

```

%%capture
!apt-get install -y -qq software-properties-common python-software-properties module-init-
!add-apt-repository -y ppa:fenics-packages/fenics
!apt-get update -qq
!apt install -y --no-install-recommends fenics
!rm -rf *
from dolfin import *

```

▼ Exercise 1

▼ Euler Explicit

```

n = 20
tMax = 1
Re = Constant(1)
dt = 0.1

u_exact = Expression((' -cos(x[0])*sin(x[1])*sin(2*t)',
                      'sin(x[0])*cos(x[1])*sin(2*t)'), t=0.0, degree=2)

f = Expression((
    '-2 * cos(x[0]) * sin(x[1]) * (cos(2 * t) + sin(2 * t) / Re)',
    '2 * sin(x[0]) * cos(x[1]) * (cos(2 * t) + sin(2 * t) / Re)'
), degree=2, t=0.0, Re=Re)

p_exact = Expression('-0.25*(cos(2*x[0])+cos(x[1]))*sin(2*t)*sin(2*t)', t=0.0, degree=2)

mesh = UnitSquareMesh(n, n, 'crossed')
Q = FiniteElement('CG', mesh.ufl_cell(), 1)
V = VectorElement('CG', mesh.ufl_cell(), 2)
X = FunctionSpace(mesh, V*Q)

u, p = TrialFunctions(X)
v, q = TestFunctions(X)

def boundary(x, on_boundary):
    return on_boundary
def origin(x, on_boundary):
    return near(x[0], 0) and near(x[1], 0)

x_ex=Expression(('u[0]', 'u[1]', 'p'), u=u_exact, p=p_exact, degree=2)
x = project(x_ex, X)
u_old, p_old = x.split()

t=0.0
while t<tMax:
    t += dt
    u_exact.t = t
    p_exact.t = t

```

```

f.t = t

bc =[DirichletBC(X.sub(0), u_exact, boundary),
     DirichletBC(X.sub(1), p_exact, origin, 'pointwise')]

a =(dot(u, v)/dt + inner(grad(u), grad(v))/Re - p*div(v) - q*div(u))*dx
l = (dot(f, v) + dot(u_old, v)/dt - dot(grad(u_old)*u_old, v))*dx

x = Function(X)
solve(a==l, x, bc)

uh, ph = x.split()
u_old = uh

eH1 = errornorm(u_exact, uh, 'H10')
eL2 = errornorm(p_exact, ph, 'L2')

print('time ' + str(t))
print('eH10 ' + str(eH1))
print('eL2 ' + str(eL2))
print()
print()

```



```
time 0.1
eH10 0.0006676566875332003
eL2 0.008394810412694088
```

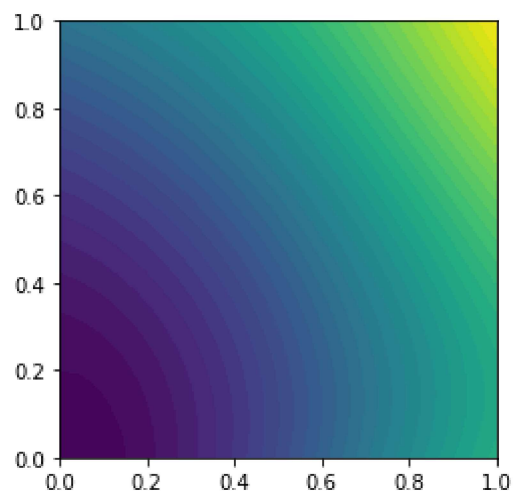
```
time 0.2
eH10 0.001737971467477484
eL2 0.018829690648151348
```

```
time 0.30000000000000004
eH10 0.002801733826571586
eL2 0.01780172973189983
```

```
time 0.4
eH10 0.00376367834377708
eL2 0.009563958406727656
```

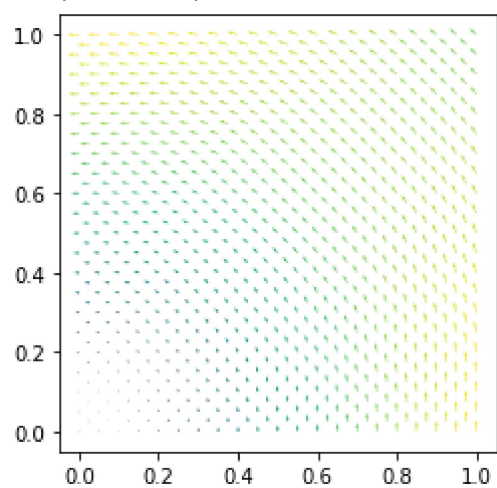
plot(ph)

↗ <matplotlib.tri.tricontour.TriContourSet at 0x7fae38134a90>



plot(uh)

↗ <matplotlib.quiver.Quiver at 0x7fae38086f98>



▼ Euler Semi-Implicit

```
n = 20
tMax = 1
Re = Constant(1)
dt = 0.1

u_exact = Expression((-cos(x[0])*sin(x[1])*sin(2*t)',
                      'sin(x[0])*cos(x[1])*sin(2*t)'), t=0.0, degree=2)

f = Expression((
    '-2 * cos(x[0]) * sin(x[1]) * (cos(2 * t) + sin(2 * t) / Re)',
    '2 * sin(x[0]) * cos(x[1]) * (cos(2 * t) + sin(2 * t) / Re)'
), degree=2, t=0.0, Re=Re)

p_exact = Expression('-0.25*(cos(2*x[0])+cos(x[1]))*sin(2*t)*sin(2*t)', t=0.0, degree=2)

mesh = UnitSquareMesh(n, n, 'crossed')
Q = FiniteElement('CG', mesh.ufl_cell(), 1)
V = VectorElement('CG', mesh.ufl_cell(), 2)
X = FunctionSpace(mesh, V*Q)

u, p = TrialFunctions(X)
v, q = TestFunctions(X)

def boundary(x, on_boundary):
    return on_boundary
def origin(x, on_boundary):
    return near(x[0], 0) and near(x[1], 0)

x_ex=Expression(('u[0]', 'u[1]', 'p'), u=u_exact, p=p_exact, degree=2)
x = project(x_ex, X)
u_old, p_old = x.split()

t=0.0
while t<tMax:
    t += dt
    u_exact.t = t
    p_exact.t = t
    f.t = t

    bc =[DirichletBC(X.sub(0), u_exact, boundary),
         DirichletBC(X.sub(1), p_exact, origin, 'pointwise')]

    a =(dot(u, v)/dt + inner(grad(u), grad(v))/Re - p*div(v) - q*div(u) + dot(grad(u)*u_old,
    l = (dot(f, v) + dot(u_old, v)/dt)*dx

    x = Function(X)
    solve(a==l, x, bc)

    uh, ph = x.split()
    u_old = uh

    eH1 = errornorm(u_exact, uh, 'H10')
    eL2 = errornorm(p_exact, ph, 'L2')
```

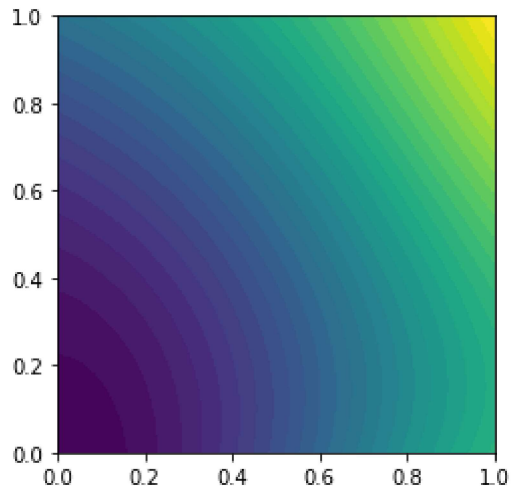
```
print('time ' + str(t))
print('eH10 ' + str(eH1))
print('eL2 ' + str(eL2))
print()
print()
```



```
time 0.1
eH10 0.0006676566875054168
```

```
plot(ph)
```

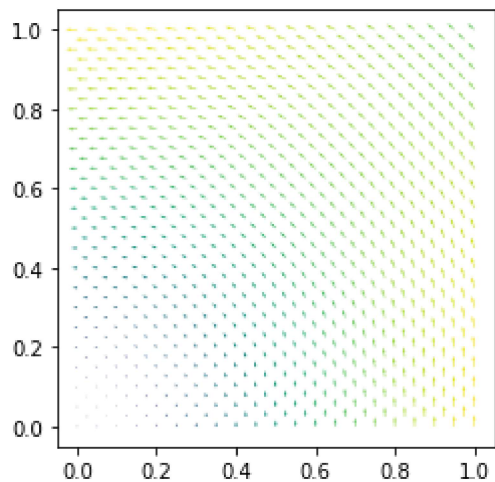
```
↳ <matplotlib.tri.tricontour.TriContourSet at 0x7fae3ba23898>
```



```
time 0.004576414031331601
```

```
plot(uh)
```

```
↳ <matplotlib.quiver.Quiver at 0x7fae3b9fada0>
```



```
time 0.8999999999999999
```

▼ Euler Implicit

```
n = 10
tMax = 1
Re = Constant(1)
dt = 0.1
tol = 1e-8
nMax = 200
```

```
u_exact = Expression((-cos(x[0])*sin(x[1])*sin(2*t)',
                    'sin(x[0])*cos(x[1])*sin(2*t)'), t=0.0, degree=2)
```

```
f = Expression((
    '-2 * cos(x[0]) * sin(x[1]) * (cos(2 * t) + sin(2 * t) / Re)',
    '2 * sin(x[0]) * cos(x[1]) * (cos(2 * t) + sin(2 * t) / Re)'
), degree=2, t=0.0, Re=Re)
```

```

p_exact = Expression('-0.25*(cos(2*x[0])+cos(x[1]))*sin(2*t)*sin(2*t)', t=0.0, degree=2)

mesh = UnitSquareMesh(n, n, 'crossed')
Q = FiniteElement('CG', mesh.ufl_cell(), 1)
V = VectorElement('CG', mesh.ufl_cell(), 2)
X = FunctionSpace(mesh, V*Q)

x_ex=Expression(('u[0]', 'u[1]', 'p'), u=u_exact, p=p_exact, degree=2)
x = project(x_ex, X)
u_old, p_old = x.split()

def solveNewImpl(bc, X, u_old, p_old, tol, nMax, f, Re, dt):
    u, p = TrialFunctions(X)
    v, q = TestFunctions(X)

    uh_old = u_old
    ph_old = p_old

    n = 0
    res = tol+1
    print('Newton Method')
    while res>tol and n<nMax:
        n += 1

        a = (dot(u, v)/dt + inner(grad(u), grad(v))/Re - p*div(v) - q*div(u) +
              dot(grad(u)*uh_old, v) + dot(grad(uh_old)*u, v))*dx
        l = (dot(f, v) + dot(u_old, v)/dt + dot(grad(uh_old)*uh_old, v))*dx

        x = Function(X)
        solve(a==l, x, bc)

        uh, ph = x.split()

        res = errornorm(uh_old, uh, 'H10')/norm(uh, 'H10') + errornorm(ph_old, ph, 'L2')/norm(
        print('It ' + str(n) + ' - residual ' + str(res))
        uh_old, ph_old = uh, ph

    return uh, ph

t=0.0
while t<tMax:
    t += dt
    print('time ' + str(t))
    u_exact.t = t
    p_exact.t = t
    f.t = t

    def boundary(x, on_boundary):
        return on_boundary
    def origin(x, on_boundary):
        return near(x[0], 0) and near(x[1], 0)

    bc =[DirichletBC(X.sub(0), u_exact, boundary),

```

```

DirichletBC(X.sub(1), p_exact, origin, 'pointwise')]

uh, ph = solveNewImpl(bc, X, u_old, p_old, tol, nMax, f, Re, dt)
u_old, p_old = uh, ph

eH1 = errornorm(u_exact, uh, 'H10')
eL2 = errornorm(p_exact, ph, 'L2')
print()
print('Timestep error')
print('eH10  ' + str(eH1))
print('eL2   ' + str(eL2))
print()
print()
print()

```



It 1 - residual 0.16054239442447188
It 2 - residual 0.003331767153803851
It 3 - residual 8.032447023917714e-13

Timestep error

eH10 0.005643211886599856
eL2 0.11046825548217691

time 0.7999999999999999

Newton Method

It 1 - residual 0.04228496665503357
It 2 - residual 0.0002261247278028909
It 3 - residual 3.195970564223893e-12

Timestep error

eH10 0.005842193164579781
eL2 0.11364918514340644

time 0.8999999999999999

Newton Method

It 1 - residual 0.0791965324570069
It 2 - residual 0.0007922595364719207
It 3 - residual 4.992854663251216e-13

Timestep error

eH10 0.005808367187546649
eL2 0.10765036667751289

time 0.9999999999999999

Newton Method

It 1 - residual 0.21415115712943328
It 2 - residual 0.005709251644365031
It 3 - residual 8.576726035472938e-13

Timestep error

eH10 0.005543108549316101
eL2 0.09347340607325126

time 1.0999999999999999

Newton Method

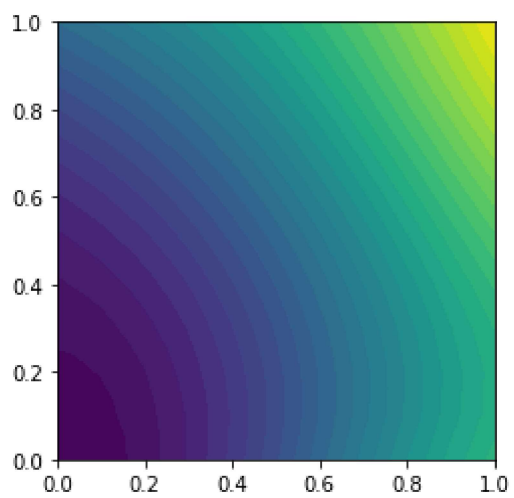
It 1 - residual 0.377994059976685
It 2 - residual 0.01755748875566241
It 3 - residual 1.2811971524378356e-12

Timestep error

eH10 0.005056989664520333
eL2 0.07349146674054102

plot(ph)

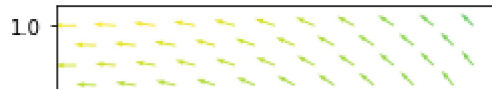
↗ <matplotlib.tri.tricontour.TriContourSet at 0x7fae37f4b2b0>



plot(uh)

↗

<matplotlib.quiver.Quiver at 0x7fae37e24320>



▼ Exercise 2



▼ Without Stabilization



```
u_top = Constant((1, 0))
u_other = Constant((0, 0))

f = Constant((0, 0))
n=40
tMax = 10
Re = Constant(10000)
dt = 0.1

mesh = UnitSquareMesh(n, n, 'crossed')

# Space Definition
V = VectorElement('CG', mesh.ufl_cell(), 2)
Q = FiniteElement('CG', mesh.ufl_cell(), 1)

X = FunctionSpace(mesh, V*Q)

# Boundary
def top(x, on_boundary):
    return near(x[1], 1)
def other(x, on_boundary):
    return on_boundary and not near(x[1], 1)
def origin(x, on_boundary):
    return near(x[0], 0) and near(x[1], 0)

bc1 = DirichletBC(X.sub(0), u_top, top)
bc2 = DirichletBC(X.sub(0), u_other, other)
bc3 = DirichletBC(X.sub(1), Constant(0), origin, 'pointwise')
bcs=[bc1, bc2, bc3]

u, p = TrialFunctions(X)
v, q = TestFunctions(X)

a = (inner(grad(u), grad(v))/Re - p*div(v) - q*div(u))*dx
l = dot(f, v)*dx

x = Function(X)
solve(a==l, x, bcs)
u, p = x.split()
ufile = File('velocity.pvd')
pfile = File('pressure.pvd')
t=0
```

```

ufile << (u, t)
pfile << (p, t)

while t<tMax:
    t += dt

    u_old, p_old = u, p

    u, p = TrialFunctions(X)
    v, q = TestFunctions(X)

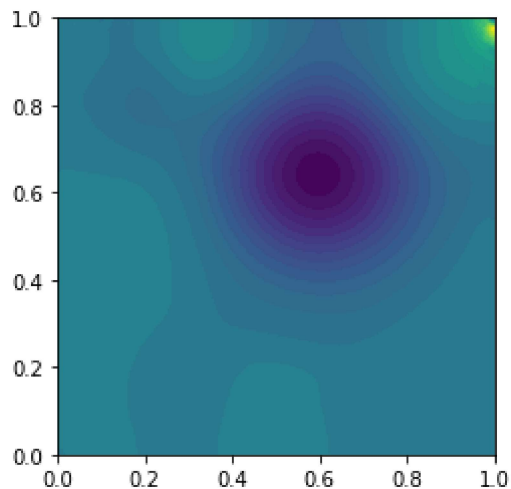
    a = (dot(u, v)/dt + inner(grad(u), grad(v))/Re + dot(grad(u)*u_old, v) - p*div(v) - q*di
    l = (dot(f, v) + dot(u_old, v)/dt)*dx

    x = Function(X)
    solve(a==l, x, bcs)
    u, p = x.split()
    ufile << (u, t)
    pfile << (p, t)

```

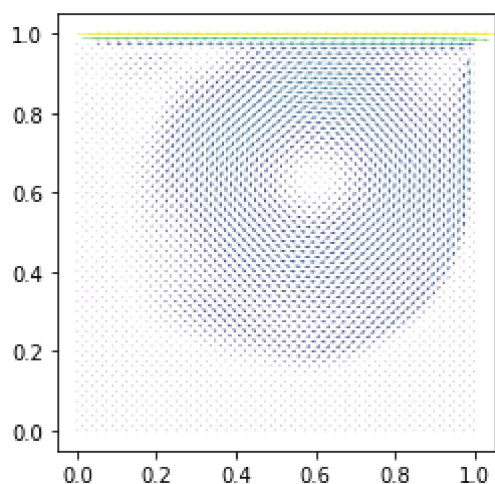
```
plot(p)
```

↳ <matplotlib.tri.tricontour.TriContourSet at 0x7fae37dcf518>



```
plot(u)
```

↳ <matplotlib.quiver.Quiver at 0x7fae372d6f28>



▼ With Stabilization

```
u_top = Constant((1, 0))
u_other = Constant((0, 0))
f = Constant((0, 0))
n=20
tMax = 10
Re = Constant(10000)
dt = 0.1

mesh = UnitSquareMesh(n, n, 'crossed')

# Space Definition
V = VectorElement('CG', mesh.ufl_cell(), 2)
Q = FiniteElement('CG', mesh.ufl_cell(), 1)

X = FunctionSpace(mesh, V*Q)

# Boundary
def top(x, on_boundary):
    return near(x[1], 1)
def other(x, on_boundary):
    return on_boundary and not near(x[1], 1)
def origin(x, on_boundary):
    return near(x[0], 0) and near(x[1], 0)

bc1 = DirichletBC(X.sub(0), u_top, top)
bc2 = DirichletBC(X.sub(0), u_other, other)
bc3 = DirichletBC(X.sub(1), Constant(0), origin, 'pointwise')
bcs=[bc1, bc2, bc3]

u, p = TrialFunctions(X)
v, q = TestFunctions(X)

a = (inner(grad(u), grad(v))/Re - p*div(v) - q*div(u))*dx
l = dot(f, v)*dx

x = Function(X)
solve(a==l, x, bcs)
u, p = x.split()
ufile = File('velocity.pvd')
pfile = File('pressure.pvd')
ufile << (u, t)
pfile << (p, t)

t=0
while t<tMax:
    t += dt

    u_old, p_old = u, p

    u, p = TrialFunctions(X)
    v, q = TestFunctions(X)
```

```

# pay attention to the sign of the last term here! In the case of stabilized FEM
# we have to put a plus sign in order to match with the skew-symmetry requirement
# of the SUPG formulation of the stabilization term... otherwise we should modify
# the signs in the stabilization terms
a = (dot(u, v)/dt + inner(grad(u), grad(v))/Re + dot(grad(u)*u_old, v) - p*div(v) + q*di
l = (dot(f, v) + dot(u_old, v)/dt)*dx

h=CellDiameter(mesh)
unorm=sqrt(dot(u_old, u_old))
Pe = unorm*h*Re/2
eps = conditional(Pe>1, 1, Pe)
tau = h*eps/(2*unorm)

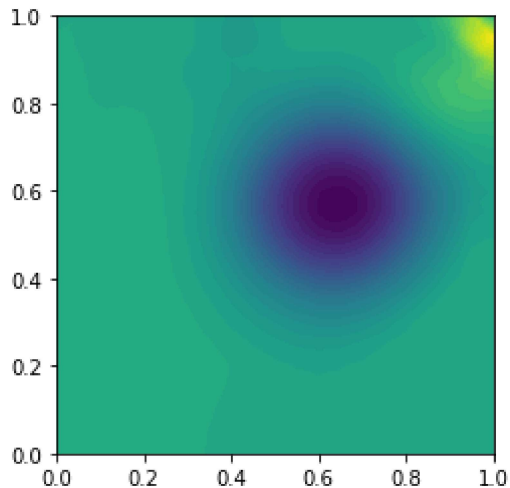
A = lambda u, p: u / Constant(dt) + (grad(u) * u_old) - div(grad(u)) / Re + grad(p)
A_SS = lambda u, p: (grad(u) * u_old) + grad(p) + 0.5 * div(u_old) * u
a += tau * (dot(A(u, p), A_SS(v, q)) + div(u) * div(v)) * dx
l += tau * (dot(u_old / Constant(dt) + f, A_SS(v, q))) * dx

x = Function(X)
solve(a==l, x, [bc1, bc2, bc3])
u, p = x.split()
ufile << (u, t)
pfile << (p, t)

```

plot(p)

↳ <matplotlib.tri.tricontour.TriContourSet at 0x7fae370bdc88>



plot(u)

↳